Ενότητα 03:
Αριθμητική Υπολογιστών

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

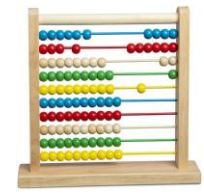
1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Outline

- Αριθμοί
- Πρόσθεση, Αφαίρεση, Πολλαπλασιασμός, Διαίρεση
- Αριθμοί Κινητής Υποδιαστολής (Floating-point numbers)

- Υλικό από Κεφάλαιο 3 του βιβλίου του μαθήματος



2

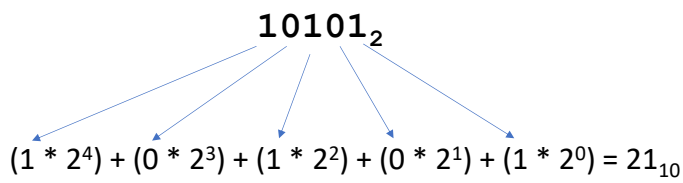
Αριθμητική Υπολογιστών

- Μια λέξη μηχανής (word) αποτελείται από ένα σύνολο δυαδικών ψηφίων (bits) → η λέξη αναπαρίσταται ως ένας δυαδικός αριθμός
  - Βάση 2: 10101_2 (21_{10})

- **Προβλήματα:**
  - Οι αριθμοί που μπορούμε να αναπαραστήσουμε είναι πεπερασμένοι
    - Πρόβλημα υπερχείλισης - overflow
      - Όταν ένας αριθμός δεν μπορεί να αναπαρασταθεί με ένα συγκεκριμένο αριθμό bits
  - Αρνητικοί αριθμοί (Signed και Unsigned Numbers)
  - Κλάσματα και πραγματικοί αριθμοί (Floating-point Numbers)

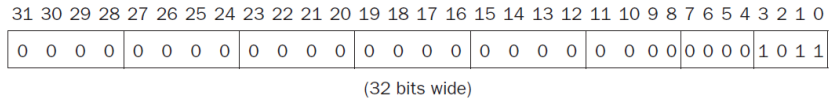
Δυαδικό σύστημα (Βάση 2)

- Πως το μετατρέπουμε από μία βάση σε άλλη?
  - Βάση 2 (δυαδικό) σε Βάση 10 (δεκαδικό)



Bit significance (in MIPS)

Most Significant Bit (MSB)



Least Significant Bit (LSB)

Συστήματα Αναπαράστασης Πρόσημου

Sign Magnitude

000 = +0
001 = +1
010 = +2
011 = +3
100 = -0
101 = -1
110 = -2
111 = -3

- Χρησιμοποιούμε 1 bit για να αναπαραστήσουμε το πρόσημο
  - Ποιο bit?
  - +/- 0

One's Complement

000 = +0
001 = +1
010 = +2
011 = +3
100 = -3
101 = -2
110 = -1
111 = -0

- Αντιστρέφουμε τα bits (από 0 σε 1 και από 1 σε 0)
  - +/- 0

Συστήματα Αναπαράστασης Πρόσημου

- Two's Complement
  - Leading 0s → positive
  - Leading 1s → negative

Όλοι οι αρνητικοί αριθμοί έχουν 1 στο Most Significant Bit (MSB) → **Sign Bit**

000 = +0
001 = +1
010 = +2
011 = +3
100 = -4
101 = -3
110 = -2
111 = -1

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} = 0_{ten}$
$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = 1_{ten}$
$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} = 2_{ten}$
...
$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} = 2{,}147{,}483{,}645_{ten}$
$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} = 2{,}147{,}483{,}646_{ten}$
$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} = 2{,}147{,}483{,}647_{ten}$
$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two} = -2{,}147{,}483{,}648_{ten}$
$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = -2{,}147{,}483{,}647_{ten}$
$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} = -2{,}147{,}483{,}646_{ten}$
...
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} = -3_{ten}$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} = -2_{ten}$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{two} = -1_{ten}$

MaxInt

MinInt: Δεν έχει αντίστοιχο θετικό

$(b_{31}*(-2^{31})) + (b_{30}*2^{30}) + \ldots + (b_1*2^1) + (b_0*2^0)$

Sign Bit

Μετατροπή Δυαδικού σε Δεκαδικό. Πώς?

Binary to Decimal Conversion

What is the decimal value of this 32-bit two's complement number?

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_{two}$

Substituting the number's bit values into the formula above:

$(1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \ldots + (1 \times 2^2) + (0 \times 2^1) + (0 \times 2^0)$
$= -2^{31} + 2^{30} + 2^{29} + \ldots + 2^2 + 0 + 0$
$= -2{,}147{,}483{,}648_{ten} + 2{,}147{,}483{,}644_{ten}$
$= -4_{ten}$

Πιο εύκολο τρόπο?

Two's Complement: Negation Shortcut

- Αντιστρέφουμε όλα τα bits KAI προσθέτουμε 1

$2_{ten} = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two}$

$$\begin{array}{rr} & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{two} \\ + & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad 1_{two} \\ \hline = & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two} \\ = & -2_{ten} \end{array}$$

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{two}$

$$\begin{array}{rr} & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} \\ + & \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad 1_{two} \\ \hline = & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{two} \\ = & 2_{ten} \end{array}$$

9

Signed and Unsigned Numbers

- Ένας μικροεπεξεργαστής πρέπει να μπορεί να χειρίζεται signed KAI unsigned αριθμούς
  - Π.χ., signed για αριθμούς, unsigned για διευθύνσεις
- Η C έχει τον τύπο δεδομένων **`int`** (για signed αριθμούς) και τον τύπο **`unsigned int`** (για θετικούς μόνο = unsigned αριθμούς)
- Θεωρήστε:

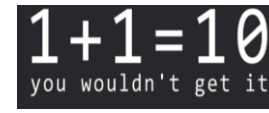
```
$s0 = 1111 1111 1111 1111 1111 1111 1111 1111
$s1 = 0000 0000 0000 0000 0000 0000 0000 0001
slt $t0, $s0, $s1       $t0 = ;
sltu $t1, $s0, $s1      $t1 = ;
$t0 = 1
$t1 = 0
```


10

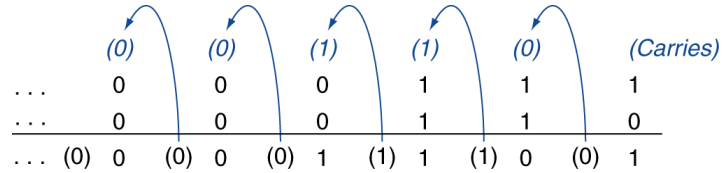
MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1,$s2,$s3	$s1 = $s2 + $s3	Three operands
	subtract	sub $s1,$s2,$s3	$s1 = $s2 - $s3	Three operands
	add immediate	addi $s1,$s2,100	$s1 = $s2 + 100	+ constant
Data transfer	load word	lw $s1,100($s2)	$s1 = Memory[$s2 + 100]	Word from memory to register
	store word	sw $s1,100($s2)	Memory[$s2 + 100] = $s1	Word from register to memory
	load half unsigned	lhu $s1,100($s2)	$s1 = Memory[$s2 + 100]	Halfword memory to register
	store half	sh $s1,100($s2)	Memory[$s2 + 100] = $s1	Halfword register to memory
	load byte unsigned	lbu $s1,100($s2)	$s1 = Memory[$s2 + 100]	Byte from memory to register
	store byte	sb $s1,100($s2)	Memory[$s2 + 100] = $s1	Byte from register to memory
	load upper immediate	lui $s1,100	$s1 = 100 * 2^{16}	Loads constant in upper 16 bits
Logical	and	and $s1,$s2,$s3	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	or $s1,$s2,$s3	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	nor $s1,$s2,$s3	$s1 = ~ ($s2 |$s3)	Three reg. operands; bit-by-bit NOR
	and immediate	andi $s1,$s2,100	$s1 = $s2 & 100	Bit-by-bit AND with constant
	or immediate	ori $s1,$s2,100	$s1 = $s2 | 100	Bit-by-bit OR with constant
	shift left logical	sll $s1,$s2,10	$s1 = $s2 << 10	Shift left by constant
	shift right logical	srl $s1,$s2,10	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	beq $s1,$s2,25	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne $s1,$s2,25	if ($s1 != $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; two's complement
	set less than immediate	slti $s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare < constant; two's complement
	set less than unsigned	sltu $s1,$s2,$s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; unsigned numbers
	set less than immediate unsigned	sltiu $s1,$s2,100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare < constant; unsigned numbers
Uncondi-tional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr $ra	go to $ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4; go to 10000	For procedure call

Πρόσθεση



- Όπως μάθαμε στο δημοτικό!
- 1 για κρατούμενο (carry)
- Παράδειγμα: 7 + 6



- **Overflow:**
  - Προσθέτουμε 2 θετικούς αριθμούς (→ Overflow εάν το sign bit είναι 1)
  - Προσθέτουμε 2 αρνητικούς αριθμούς (→ Overflow εάν το sign bit είναι 0)
  - Προσθέτουμε ένα θετικό και ένα αρνητικό αριθμό (→ NO Overflow)

Αφαίρεση

- Η αφαίρεση υλοποιείται με πρόσθεση αρνητικών αριθμών (two's complement)
- Παράδειγμα: 7 – 6 = 7 + (-6)

$$\begin{array}{lll} & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{two} & = 7_{ten} \\ + & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1010_{two} & = -6_{ten} \\ \hline = & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} & = 1_{ten} \end{array}$$

- **Overflow:**
  - Αφαιρούμε θετικό από αρνητικό αριθμό (→ Overflow εάν το sign bit είναι 0)
  - Αφαιρούμε αρνητικό από θετικό αριθμό (→ Overflow εάν το sign bit είναι 1)
  - Αφαιρούμε 2 θετικούς ή 2 αρνητικούς αριθμούς (→ NO Overflow)

Overflow conditions summary

Operation	Operand A	Operand B	Result indicating overflow
$A + B$	≥ 0	≥ 0	< 0
$A + B$	< 0	< 0	≥ 0
$A - B$	≥ 0	< 0	< 0
$A - B$	< 0	≥ 0	≥ 0

FIGURE 3.3 Overflow conditions for addition and subtraction.

Πως χειριζόμαστε το Overflow?

- Exception/Interrupt:
  - Από εντολές MIPS: **`add, addi, sub`**
  - Η διεύθυνση της εντολής που ανίχνευσε την διακοπή φυλάγεται για πιθανή επανάληψη των εντολών που διακόπηκαν
    - Καταχωρητής **`$epc`** (exception program counter)
  - Η κανονική ροή του προγράμματος διακόπτεται και προχωρεί σε μια προκαθορισμένη διεύθυνση για να εκτελέσει το «exception routine»
  - **`mfc0`** (move from system control) για επιστροφή στη κανονική ροή
- Οι λεπτομέρειες εξαρτώνται από το λογισμικό, γλώσσα προγραμματισμού, κλπ.
- Κάποιες γλώσσες προγραμματισμού αγνοούν το Overflow (π.χ., C)
  - Χρησιμοποιεί τις εντολές MIPS **`addu, addui, subu`**

MIPS assembly language

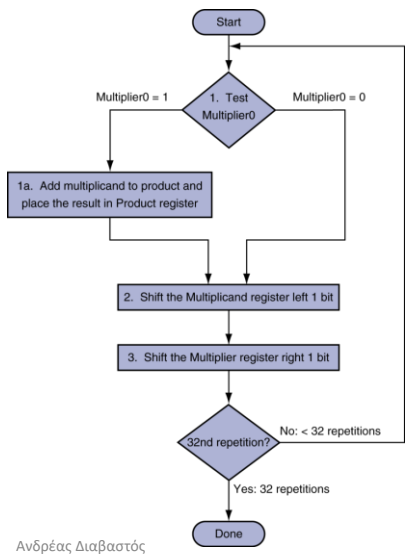
Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three operands; overflow detected
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 - $s3	Three operands; overflow detected
	add immediate	`addi $s1,$s2,100`	$s1 = $s2 + 100	+ constant; overflow detected
	add unsigned	`addu $s1,$s2,$s3`	$s1 = $s2 + $s3	Three operands; overflow undetected
	subtract unsigned	`subu $s1,$s2,$s3`	$s1 = $s2 - $s3	Three operands; overflow undetected
	add immediate unsigned	`addiu $s1,$s2,100`	$s1 = $s2 + 100	+ constant; overflow undetected
	move from coprocessor register	`mfc0 $s1,$epc`	$s1 = $epc	Used to copy Exception PC plus other special registers
Data transfer	load word	`lw $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Word from memory to register
	store word	`sw $s1,100($s2)`	Memory[$s2 + 100] = $s1	Word from register to memory
	load half unsigned	`lhu $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Halfword memory to register
	store half	`sh $s1,100($s2)`	Memory[$s2 + 100] = $s1	Halfword register to memory
	load byte unsigned	`lbu $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Byte from memory to register
	store byte	`sb $s1,100($s2)`	Memory[$s2 + 100] = $s1	Byte from register to memory
	load upper immediate	`lui $s1,100`	$s1 = 100 * 2^{16}	Loads constant in upper 16 bits
Logical	and	`and $s1,$s2,$s3`	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	$s1 = ~ ($s2 |$s3)	Three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,100`	$s1 = $s2 & 100	Bit-by-bit AND with constant
	or immediate	`ori $s1,$s2,100`	$s1 = $s2 | 100	Bit-by-bit OR with constant
	shift left logical	`sll $s1,$s2,10`	$s1 = $s2 << 10	Shift left by constant
	shift right logical	`srl $s1,$s2,10`	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	`beq $s1,$s2,25`	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	`bne $s1,$s2,25`	if ($s1 != $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; two's complement
	set less than immediate	`slti $s1,$s2,100`	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare < constant; two's complement
	set less than unsign	`sltu $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; unsigned
	set less than immediate unsigned	`sltiu $s1,$s2,100`	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare < constant; unsigned
Uncondi-tional jump	jump	`j 2500`	go to 10000	Jump to target address
	jump register	`jr $ra`	go to $ra	For switch, procedure return
	jump and link	`jal 2500`	$ra = PC + 4; go to 10000	For procedure call

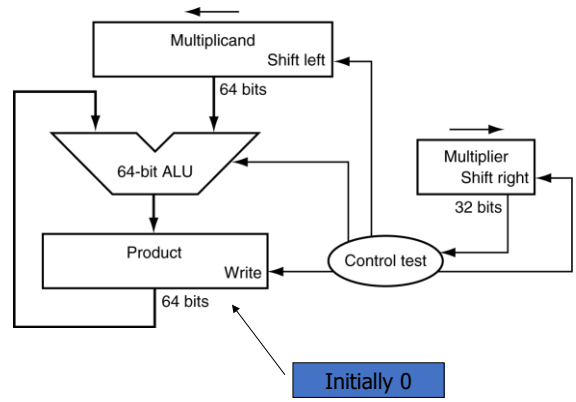
Πολλαπλασιασμός

- Πιο πολύπλοκος από πρόσθεση ή αφαίρεση
  - Υλοποιείται με ολίσθηση και πρόσθεση (shift-and-add)
- Χρειάζεται περισσότερο χρόνο και χώρο
- Θυμηθείτε τον αλγόριθμο από το δημοτικό:

```
    0010   (πολλαπλασιαστέος / multiplicand)
  x 1011   (πολλαπλασιαστής / multiplier)
  ------
    0010
   0010
  0000
+ 0010
  -------
  0010110  (αποτέλεσμα / product)
```

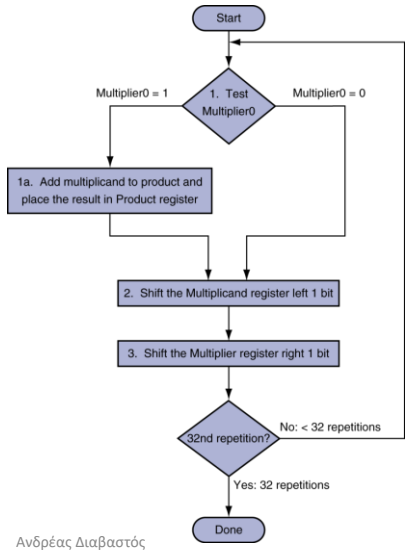

Υλοποίηση Πολλαπλασιαστή

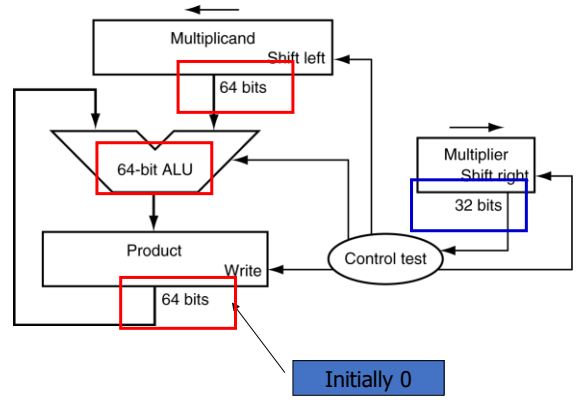




- Ο πολλαπλασιασμός εκτελείται με μια σειρά από shifts και adds των 64-bit
- Απλό αλλά αργό

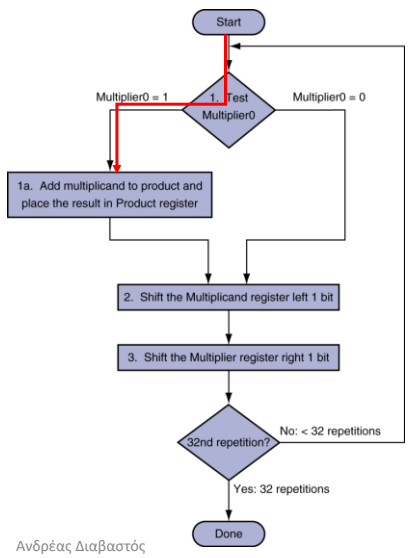
Υλοποίηση Πολλαπλασιαστή

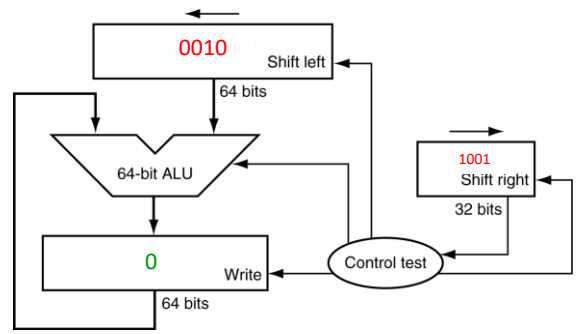




- Ο πολλαπλασιασμός εκτελείται με μια σειρά από ολισθήσεις και προσθέσεις των 64-bit
- Απλό αλλά αργό

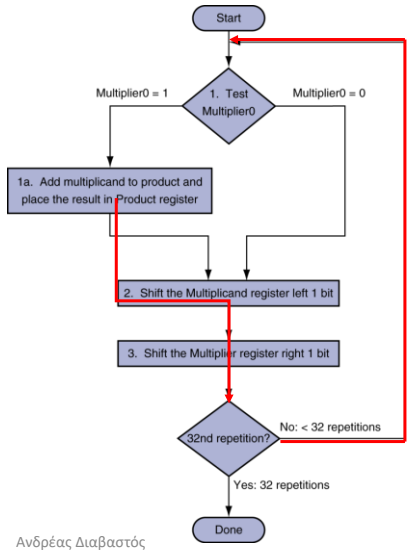
Παράδειγμα Πολλαπλασιαστή (1)

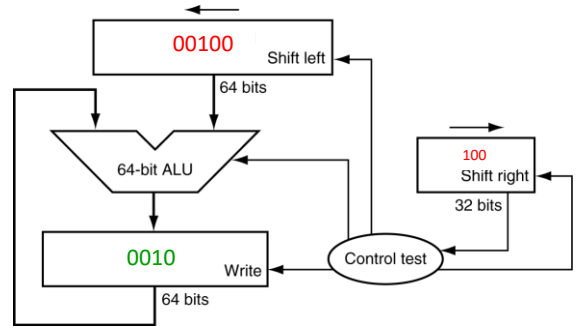




$$
\begin{array}{rr}
 & 0010 \\
\times & 1001 \\
 & 0010 \\
 & 0000 \\
 & 0000 \\
+ & 0010 \\
\hline
 & 0010010
\end{array}
$$

Παράδειγμα Πολλαπλασιαστή (2)

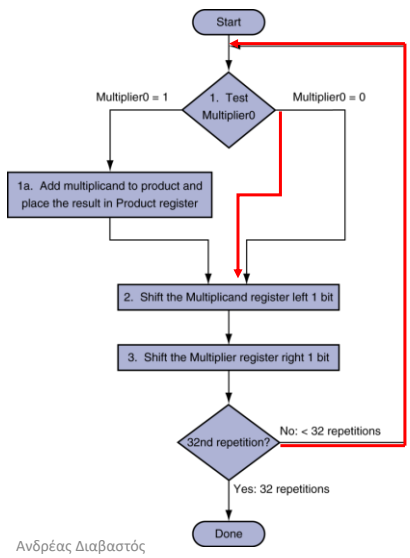


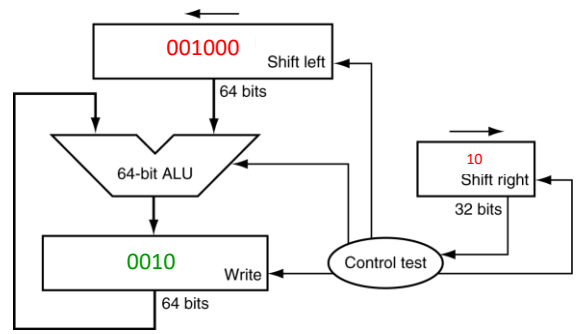


$$\begin{array}{r} 0010 \\ \times\ 1001 \\ \hline 0010 \\ 0000 \\ 0000 \\ +\ 0010 \\ \hline 0010010 \end{array}$$

21

Παράδειγμα Πολλαπλασιαστή (3)

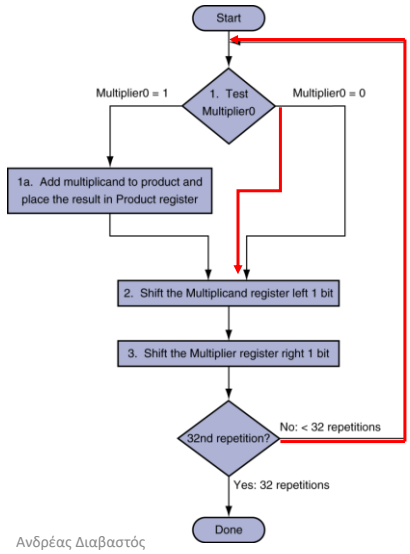


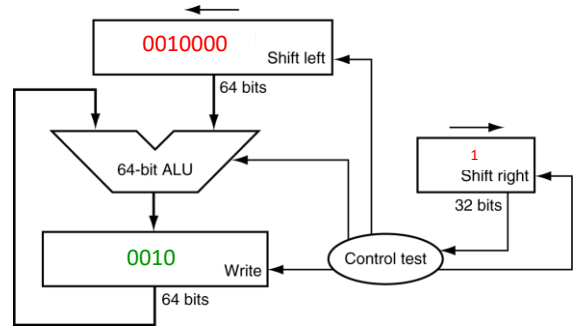


$$\begin{array}{r} 0010 \\ \times\ 1001 \\ \hline 0010 \\ 0000 \\ 0000 \\ +\ 0010 \\ \hline 0010010 \end{array}$$

22

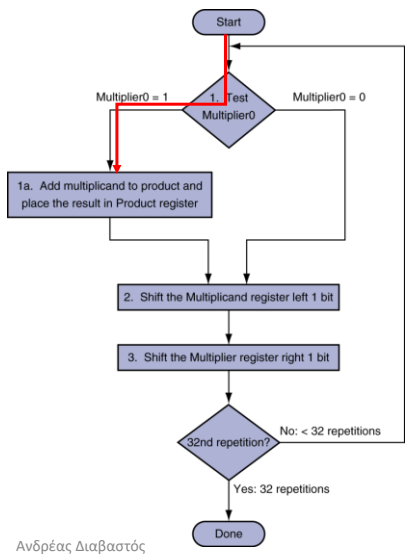
Παράδειγμα Πολλαπλασιαστή (4)

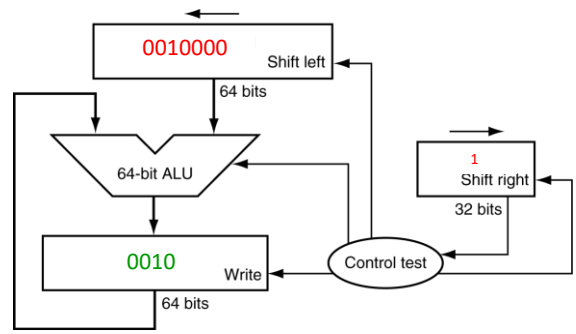




$$
\begin{array}{r}
0010 \\
\times\ 1001 \\
\hline
0010 \\
0000 \\
0000 \\
+\ 0010 \\
\hline
0010010
\end{array}
$$

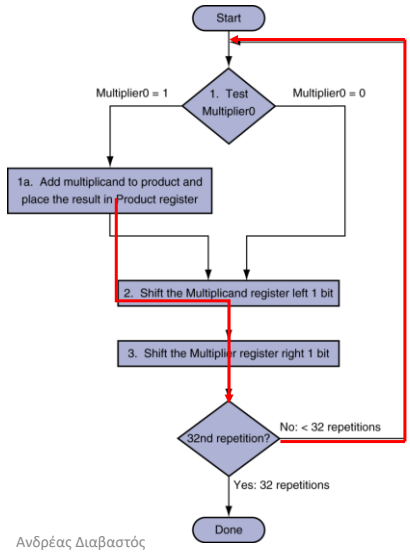
Παράδειγμα Πολλαπλασιαστή (5-1)

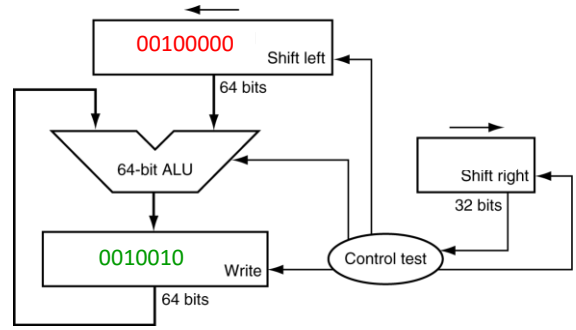




$$
\begin{array}{r}
0010 \\
\times\ 1001 \\
\hline
0010 \\
0000 \\
0000 \\
+\ 0010000 \\
\hline
0010010
\end{array}
$$

Παράδειγμα Πολλαπλασιαστή (5-2)

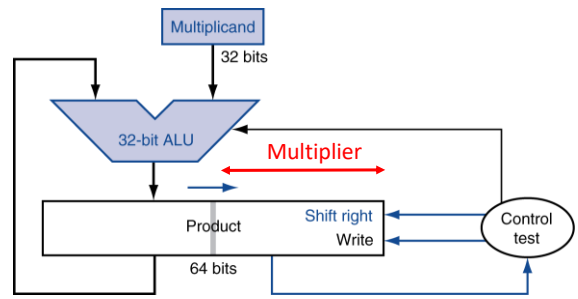




$$
\begin{array}{r}
0010 \\
\times\ 1001 \\
\hline
0010 \\
0000 \\
0000 \\
+\ 0010000 \\
\hline
0010010
\end{array}
$$

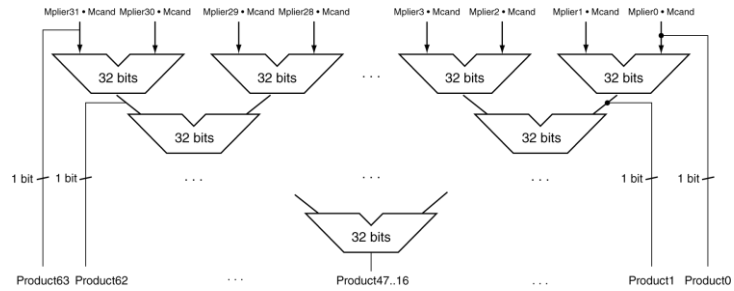
Optimized Multiplier

- Απαιτεί 1 κύκλο ρολογιού για κάθε βήμα
- Οι πράξεις εκτελούνται παράλληλα: shift για τον πολ/στη και τον πολ/στέο, ενώ ο πολ/στέος προστίθεται στο αποτέλεσμα
- Ο πολ/στής (Multiplier) ξεκινά στο δεξί μισό του αποτελέσματος (Product)



Faster Multiplier

- Χρησιμοποιούμε πολλαπλούς adders
- Pipelined → Πολλούς πολλαπλασιασμούς παράλληλα



Πολλαπλασιασμός Αρνητικών Αριθμών

- Χρειάζεται αλλαγές ο πιο πάνω αλγόριθμος?
- ΟΧΙ, πρέπει απλά να θυμόμαστε το πρόσημο
  - Αν θα είναι αρνητικός ο αριθμός τότε κάνουμε 31 επαναλήψεις shift-add και το τελευταίο το κρατάμε για το πρόσημο

Πολλαπλασιασμός στη MIPS

- Θέλουμε 2 32-bit registers για να κρατήσουμε το αποτέλεσμα
  - $Hi: most-significant 32 bits
  - $Lo: least-significant 32 bits (συνήθως αυτό χρειαζόμαστε μόνο)
    - Οι εντολές αγνοούν το overflow. Και αφήνεται στο πρόγραμμα/προγραμματιστή να ελέγξει κατά πόσον υπάρχει overflow
- Εντολές:
  - `mult rs, rt` `#signed`
    - $Hi,$Lo = $rs x $rt, 64-bit Product στους $Hi,$Lo
  - `multu rs, rt` `#un-signed`
    - $Hi,$Lo = $rs x $rt, 64-bit Product στους $Hi,$Lo
  - `mfhi rd`
    - $rd = $Hi, get a copy of Hi
  - `mflo rd`
    - $rd = $Lo, get a copy of Lo

Διαίρεση



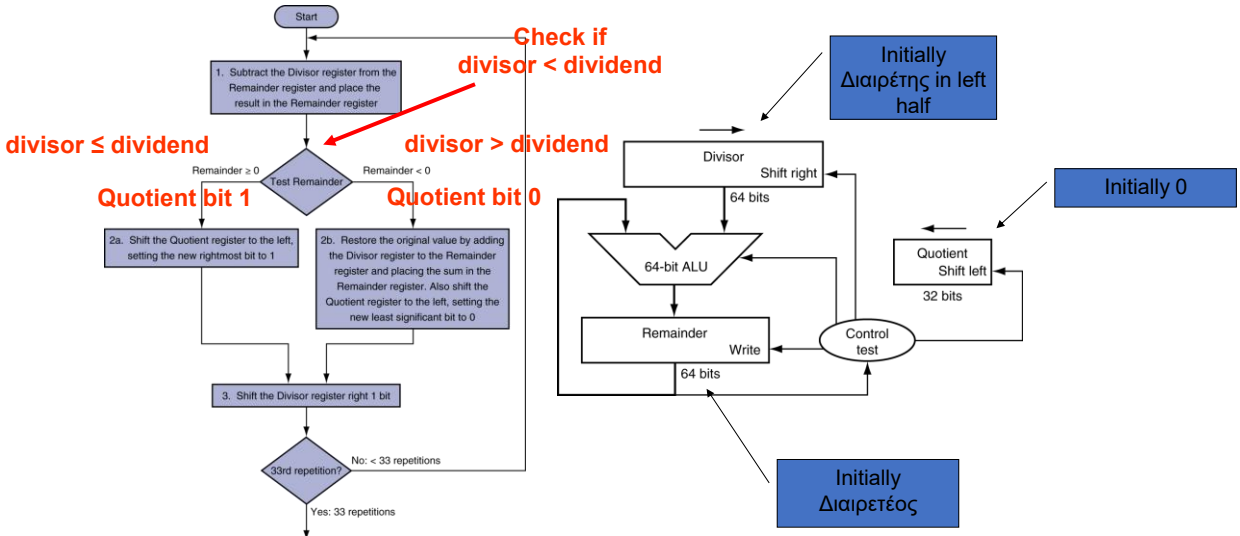
- Πιο πολύπλοκη και από πολ/σμό
- Πρέπει να χειρίζεται διαίρεση με το 0
- Και πάλι, θυμηθείτε τον αλγόριθμο του δημοτικού:

	54	1001	Quotient (Πηλίκο)
Divisor	3 | 163	1000 | 1001010	Dividend (Διαιρετέος)
(Διαιρέτης)	-15	-1000	
	13	10	
	-12	101	
	1	1010	
		-1000	
		10	Remainder(Υπόλοιπο)

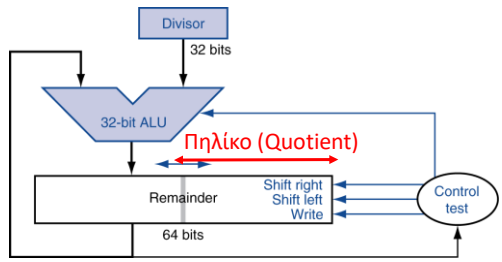
Dividend* = *Quotient* x *Divisor* + *Remainder

163 / 3 = 54x3 +1, 1,001,010 / 1000 = 1001x1000 + 10

Αλγόριθμος Διαίρεσης και Υλοποίηση



Optimized Divider



- Shift and Sub εκτελούνται παράλληλα
- 1 κύκλο για κάθε μερικό αποτέλεσμα
- Μοιάζει με τον Multiplier που είδαμε προηγουμένως
  - Το ίδιο hardware μπορείς να χρησιμοποιηθεί και για τις δύο πράξεις

Διαίρεση στη MIPS

- Θέλουμε 2 32-bit registers για να κρατήσουμε το αποτέλεσμα
  - $Hi: most-significant 32 bits για το υπόλοιπο
  - $Lo: least-significant 32 bits για το πηλίκο
- Εντολές:
  - `div rs, rt` `#signed`
    - Quotient στον $Lo = $rs / $rt, Remainder στον $Hi = $rs mod $rt
  - `divu rs, rt` `#un-signed`
    - Quotient στον $Lo = $rs / $rt, Remainder στον $Hi = $rs mod $rt
  - `mfhi rd`
    - $rd = $Hi, get a copy of Hi
  - `mflo rd`
    - $rd = $Lo, get a copy of Lo
  - Δεν έχουμε overflow ή divide-by-0 έλεγχο
    - Το πρόγραμμα πρέπει να ελέγξει για αυτό

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three operands; overflow detected
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three operands; overflow detected
	add immediate	`addi $s1,$s2,100`	$s1 = $s2 + 100	+ constant; overflow detected
	add unsigned	`addu $s1,$s2,$s3`	$s1 = $s2 + $s3	Three operands; overflow undetected
	subtract unsigned	`subu $s1,$s2,$s3`	$s1 = $s2 – $s3	Three operands; overflow undetected
	add immediate unsigned	`addiu $s1,$s2,100`	$s1 = $s2 + 100	+ constant; overflow undetected
	move from coprocessor register	`mfc0 $s1,$epc`	$s1 = $epc	Copy Exception PC + special regs
	multiply	`mult $s2,$s3`	Hi, Lo = $s2 × $s3	64-bit signed product in Hi, Lo
	multiply unsigned	`multu $s2,$s3`	Hi, Lo = $s2 × $s3	64-bit unsigned product in Hi, Lo
	divide	`div $s2,$s3`	Lo = $s2 / $s3, Hi = $s2 mod $s3	Lo = quotient, Hi = remainder
	divide unsigned	`divu $s2,$s3`	Lo = $s2 / $s3, Hi = $s2 mod $s3	Unsigned quotient and remainder
	move from Hi	`mfhi $s1`	$s1 = Hi	Used to get copy of Hi
	move from Lo	`mflo $s1`	$s1 = Lo	Used to get copy of Lo
Data transfer	load word	`lw $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Word from memory to register
	store word	`sw $s1,100($s2)`	Memory[$s2 + 100] = $s1	Word from register to memory
	load half unsigned	`lhu $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Halfword memory to register
	store half	`sh $s1,100($s2)`	Memory[$s2 + 100] = $s1	Halfword register to memory
	load byte unsigned	`lbu $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Byte from memory to register
	store byte	`sb $s1,100($s2)`	Memory[$s2 + 100] = $s1	Byte from register to memory
	load upper immediate	`lui $s1,100`	$s1 = 100 × 2^{16}	Loads constant in upper 16 bits
Logical	and	`and $s1,$s2,$s3`	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	$s1 = ~ ($s2 |$s3)	Three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,100`	$s1 = $s2 & 100	Bit-by-bit AND with constant
	or immediate	`ori $s1,$s2,100`	$s1 = $s2 | 100	Bit-by-bit OR with constant
	shift left logical	`sll $s1,$s2,10`	$s1 = $s2 << 10	Shift left by constant
	shift right logical	`srl $s1,$s2,10`	$s1 = $s2 >> 10	Shift right by constant
Condi-tional branch	branch on equal	`beq $s1,$s2,25`	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	`bne $s1,$s2,25`	if ($s1 != $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; two's complement
	set less than immediate	`slti $s1,$s2,100`	if ($s2 < 100) $s1 = 1; else $s1=0	Compare < constant; two's complement
	set less than unsigned	`sltu $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1=0	Compare less than; natural numbers
	set less than immediate unsigned	`sltiu $s1,$s2,100`	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare < constant; natural numbers
Uncondi-tional jump	jump	`j 2500`	go to 10000	Jump to target address
	jump register	`jr $ra`	go to $ra	For switch, procedure return
	jump and link	`jal 2500`	$ra = PC + 4; go to 10000	For procedure call

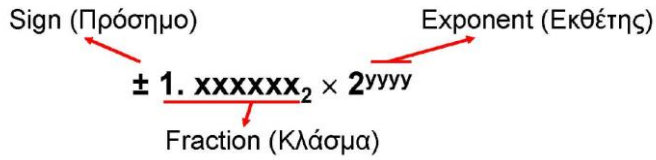
35

Αριθμητική Κινητής Υποδιαστολής (Floating-Point Arithmetic)

- Χρειαζόμαστε ένα τρόπο για αναπαράσταση:
  - κλασματικών αριθμών, π.χ., 3.14159265…
  - πολύ μικρών αριθμών, π.χ., 0.000000001 ή 1.0×10^{-9}
  - πολύ μεγάλων αριθμών, π.χ., 3,155,760,000 ή 3.15576×10^{9}
- Επιστημονικός Συμβολισμός (*Scientific Notation*)
  - $a \times 10^{n}$
    - a → πραγματικός αριθμός (συνήθως) από 1-10 (excl. 10)
    - n → ακέραιος (θετικός ή αρνητικός). Δείχνει πόσες θέσεις μετακινείται το δεκαδικό σημείο
  - Τα τελευταία δύο παραδείγματα πιο πάνω φαίνονται είναι σε μορφή επιστημονικού συμβολισμού
- Κανονικοποιημένη Τιμή (*Normalized Value*)
  - Αριθμός σε μορφή επιστημονικού συμβολισμού, αλλά χωρίς μηδενικά μπροστά
  - Π.χ., 1.0×10^{-9} είναι κανονικοποιημένο ενώ το 0.10×10^{-8} δεν είναι

36

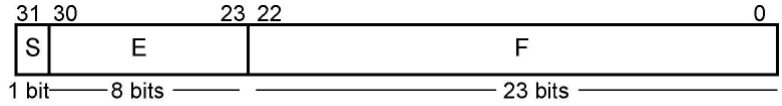
Περιγραφή Δυαδικών με Κινητή Υποδιαστολή



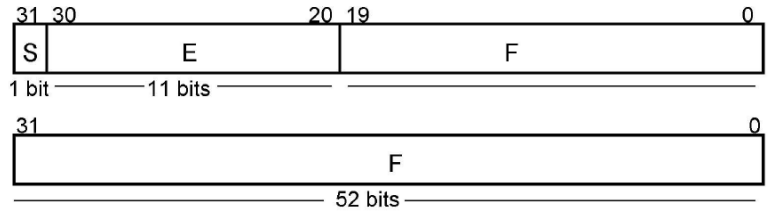
- 3 μέρη: Πρόσημο (S), Κλάσμα (F) και Εκθέτης (E)
- Γενική Μορφή: **$(-1)^S \times F \times 2^E$**
- Το μέγεθος του F καθορίζει την ακρίβεια (*accuracy or precision)*
- Το μέγεθος του E καθορίζει το εύρος (*range)* των τιμών
- Μέγεθος του F+E είναι σταθερό, άρα πρέπει να αποφασίσουμε μεταξύ F και E!
- Χρησιμοποιούμε Normalized Scientific αναπαράσταση!

Κινητή Υποδιαστολή στη MIPS

- Αναπαράσταση *sign-and-magnitude* (όχι two's complement)
- Single-precision (float in C)



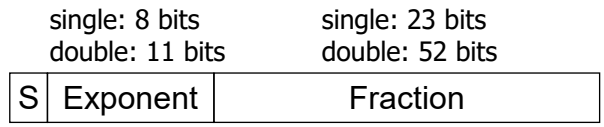
- Double-precision (double in C)



IEEE 754 - Floating Point Standard

- Ορίζεται από το IEEE Std 754-1985
- Καθορίστηκε για να λυθούν προβλήματα απόκλισης και αναπαράστασης πραγματικών αριθμών
- Χρησιμοποιείται σχεδόν από όλους του υπολογιστές που σχεδιάστηκαν μετά το 1980, ως πρότυπο αριθμητικής για κινητή υποδιαστολή
- Δύο αναπαραστάσεις
  - Single precision (32-bit)
  - Double precision (64-bit)

IEEE 754 - Floating Point Format



$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- S: sign bit (0 $\Rightarrow$ θετικός, 1 $\Rightarrow$ αρνητικός)
- Normalized Significand («1.» + Fraction bits):
  - Αφού είναι normalized, σημαίνει ότι πάντα ξεκινούμε από 1
  - Συνεπώς δεν χρειάζεται να το αναπαραστήσουμε (κερδίζουμε και 1 bit)
- Exponent: Biased Notation
  - Αποθηκεύεται σαν unsigned αριθμός (πιο εύκολο από το two's complement)
  - Single-Bias = 127, Double-Bias = 1023
  - Αν το Exponent είναι μεγαλύτερο από το Bias → θετικός εκθέτης
  - Αν το Exponent είναι μικρότερο από το Bias → αρνητικός εκθέτης

Single-Precision Range

- Exponents 00000000 και 11111111 δεσμευμένα από το hardware
  - Για Underflow και Overflow ελέγχους
- Μικρότερη τιμή:
  - Exponent: 00000001
    ⇒ actual exponent = 1 – 127 = –126
  - Fraction: 000…00 ⇒ significand = 1.00000000
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Μεγαλύτερη τιμή:
  - Exponent: 11111110
    ⇒ actual exponent = 254 – 127 = +127
  - Fraction: 111…11 ⇒ significand = $1.11111\ldots11_2 \approx 2.0_{10}$
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

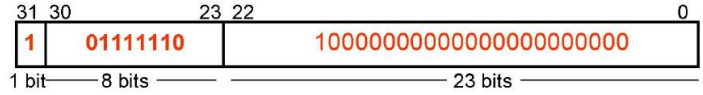
Double-Precision Range

- Exponents 0000…00 και 1111…11 δεσμευμένα από το hardware
  - Για Underflow και Overflow ελέγχους
- Μικρότερη τιμή:
  - Exponent: 00000000001
    ⇒ actual exponent = 1 – 1023 = –1022
  - Fraction: 000…00 ⇒ significand = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Μεγαλύτερη τιμή:
  - Exponent: 11111111110
    ⇒ actual exponent = 2046 – 1023 = +1023
  - Fraction: 111…11 ⇒ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Παράδειγμα Floating Point 1

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- Έχουμε τον αριθμό: $-0.75_{10} = -0.11_2$
- Scientific Notation: $-0.11_2 \times 2^0$
- Normalized Scientific Notation: $-1.1_2 \times 2^{-1}$ → $(-1)^1 \times 1.1_2 \times 2^{-1}$ IEEE 754
- Πρόσημο S = 1
- Fraction = $100\ldots00_2$
- Biased_Exponent = -1 (== Exponent - Bias)
- Bias = 127
- Exponent = Biased_Exponent + Bias = -1 + 127 = 126



Παράδειγμα Floating Point 2

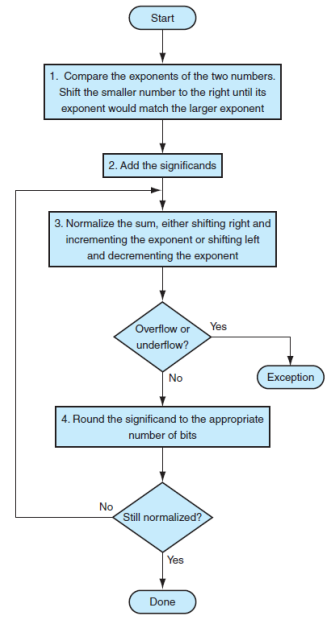
- Έχουμε τον αριθμό: 11000000101000…00

$$x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- Πρόσημο S = 1
- Fraction = $01000\ldots00_2$
- Exponent = $10000001_2 = 129$
- $x = (-1)^1 \times (1 + 01_2) \times 2^{(129 - 127)}$
  $= (-1) \times 1.25 \times 2^2$
  $= -5.0$

Πρόσθεση Floating Point

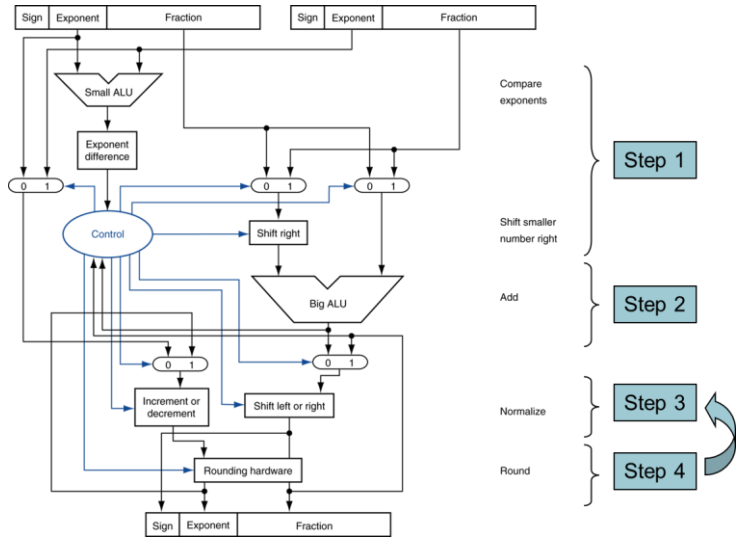
- Παράδειγμα με 4-ψήφιο (significand) δεκαδικό
  - $9.999 \times 10^{1} + 1.610 \times 10^{-1}$
- **1ο Βήμα:**
  - Συγκρίνουμε τα exponents και κάνουμε Shift δεξιά τον μικρότερο αριθμό μέχρι τα exponents να γίνουν τα ίδια
  - Έχουμε μόνο 4 ψηφία για το significand!
  - $9.999 \times 10^{1} + 0.016 \times 10^{1}$
- **2ο Βήμα:**
  - Προσθέτουμε τα Significands
  - $9.999 \times 10^{1} + 0.016 \times 10^{1} = 10.015 \times 10^{1}$
- **3ο Βήμα:**
  - Normalize τον αριθμό με Shift δεξιά και αύξηση του exponent
  - $10.015 \times 10^{1} \rightarrow 1.0015 \times 10^{2}$
- **4ο Βήμα:**
  - Στρογγυλοποίηση αριθμού σε 4 ψηφία (και Normalize εάν χρειάζεται)
  - $10.015 \times 10^{2} \rightarrow 1.002 \times 10^{2}$



Υλοποίηση FP Adder Hardware

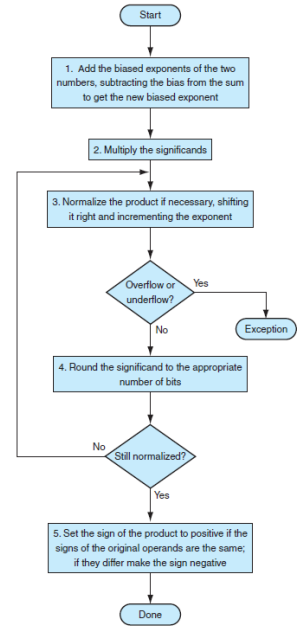
- Πολύ πολύπλοκη σε σχέση με Integer Adder
- Περισσότερους κύκλους από ότι ο Integer Adder
- Συνήθως είναι ξεχωριστό unit (coprocessor)
  - Δικές του εντολές
  - Δικούς του καταχωρητές

Υλοποίηση FP Adder Hardware



Πολλαπλασιασμός Floating Point

- Παράδειγμα με 4-ψήφιο (significand) δεκαδικό
  - $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$
- **1ο Βήμα:**
  - Προσθέτουμε τα exponents
  - Νέο Exponent = 10 + (–5) = 5
- **2ο Βήμα:**
  - Πολλαπλασιάζουμε τα Significands
  - $1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^{5}$
- **3ο Βήμα:**
  - Normalize τον αριθμός με Shift δεξιά και αύξηση του exponent
  - $10.212 \times 10^{5} \rightarrow 1.0212 \times 10^{6}$
- **4ο Βήμα:**
  - Στρογγυλοποίηση αριθμού σε 4 ψηφία (και Normalize εάν χρειάζεται)
  - $1.0212 \times 10^{6} \rightarrow 1.021 \times 10^{6}$
- **5ο Βήμα:**
  - Υπολογισμός πρόσημου από τα πρόσημα των δύο operands
  - $+1.021 \times 10^{6}$



Εντολές Floating Point για MIPS

- 32 νέοι καταχωρητές για πράξεις αριθμητικής με κινητή υποδιαστολή: **`$f0, $f1, $f2, … , $f31`** (χρησιμοποιούνται σε ζευγάρια για διπλή ακρίβεια)
  - Double-precision: **`$f0`**/**`$f1`**, **`$f2`**/**`$f3`**, …
- Πρόσθεση / Αφαίρεση / Πολ/σμός / Διαίρεση
  - `add.s $f0, $f4, $f6` # πρόσθεση, μονής ακρίβειας
  - `add.d $f0, $f4, $f6` # πρόσθεση, διπλής ακρίβειας
  - `sub.s $f0, $f4, $f6` # αφαίρεση, μονής ακρίβειας
  - `sub.d $f0, $f4, $f6` # αφαίρεση, διπλής ακρίβειας
  - `mul.s $f0, $f4, $f6` # πολ/σμός, μονής ακρίβειας
  - `mul.d $f0, $f4, $f6` # πολ/σμός, διπλής ακρίβειας
  - `div.s $f0, $f4, $f6` # διαίρεση, μονής ακρίβειας
  - `div.d $f0, $f4, $f6` # διαίρεση, διπλής ακρίβειας

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Εντολές Floating Point για MIPS

- Σύγκριση
  - `c.eq.s $f2, $f4` # σύγκριση *ισότητας*, μονής ακρίβειας
    # θέτει `cond=1` εάν True, `cond=0` εαν False → **`condition code bit`**
  - `c.eq.d $f2, $f4` # σύγκριση *ισότητας*, διπλής ακρίβειας
  - `c.neq.s $f2, $f4` # σύγκριση *ανισότητας*, μονής ακρίβειας
  - `c.neq.d $f2, $f4` # σύγκριση *ανισότητας*, διπλής ακρίβειας
  - `c.lt.s $f2, $f4` # σύγκριση για *λιγότερο από*, μονής ακρίβειας
  - `c.lt.d $f2, $f4` # σύγκριση για *λιγότερο από*, διπλής ακρίβειας
  - `c.le.s $f2, $f4` # σύγκριση για *λιγότερο από ή ίσα*, μονής ακρίβειας
  - `c.le.d $f2, $f4` # σύγκριση για *λιγότερο από ή ίσα*, διπλής ακρίβειας
  - `c.gt.s $f2, $f4` # σύγκριση για *μεγαλύτερο από*, μονής ακρίβειας
  - `c.gt.d $f2, $f4` # σύγκριση για *μεγαλύτερο από*, διπλής ακρίβειας
  - `c.ge.s $f2, $f4` # σύγκριση για *μεγαλύτερο από ή ίσα*, μονής ακρίβειας
  - `c.ge.d $f2, $f4` # σύγκριση για *μεγαλύτερο από ή ίσα*, διπλής ακρίβειας

Αριθμητική Ακριβείας

- **ΠΡΟΣΟΧΗ!!!**
  - **Αποφεύγουμε συγκρίσεις με πραγματικούς αριθμούς!!!**
- Σε αναπαραστάσεις κινητής υποδιαστολής, ο αριθμός δίνεται κατά προσέγγιση του πραγματικού αριθμού. Αυτό συμβαίνει λόγω της περιορισμένης / πεπερασμένης ακρίβειας

Εντολές Floating Point για MIPS

- c1 = coprocessor 1, μαθηματικός συνεπεξεργαστής (FP accelerator)
- Branches:
  - `bc1t TargetLabel` # Εάν cond==1 (True) διακλάδωσε στη διεύθυνση
  - `bc1f TargetLabel` # Εάν cond==0 (False) διακλάδωσε στη διεύθυνση
  - Χρησιμοποιείται με εντολές συγκρίσεως
- Load / Store:
  - `lwc1 $f1, 100($s2)` # f1 = Memory[s2+100]
  - `swc1 $f1, 100($s2)` # Memory[s2+100] = f1
- Καταχωρητής βάσεως (base register) μένει ακέραιος!
- Παράδειγμα για φόρτωση 2 αριθμών μονής ακρίβειας, πρόσθεση και αποθήκευση του αποτελέσματος στη μνήμη:

```
lwc1 $f4, x($sp)  # f4 = Memory[sp+ x]
lwc1 $f6, y($sp)  # f6 = Memory[sp+ y]
add.s $f2, $f4, $f6 # f2 = f4 + f6
swc1 $f2, z($sp)  # Memory[sp+ z] = f2
```

Παράδειγμα: C → MIPS

- Απλό πρόγραμμα για την μετατροπή της θερμοκρασίας από Fahrenheit σε Celsius:
  - fahr → $f12, result → $f0, literals in global memory space

```
float f2c (float fahr)
      { return ( (5.0/9.0) * (fahr-32.0) ); }

      lwc1 $f16, const5($gp)    # f16 = 5
      lwc1 $f18, const9($gp)    # f18 = 9
      div.s $f16, $f16, $f18    # f16 = 5/9
      lwc1 $f18, const32($gp)   # f18 = 32
      sub.s $f18, $f12, $f18    # f18 = fahr-32
      mul.s $f0, $f16, $f18     # f0 = (5/9)*(fahr-32)
      jr $ra                    # επιστρέφει
```


Ενότητα 03: Αριθμητική Υπολογιστών

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy